Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre

A Project Report on

"BusLedger"

[Code No.: COMP 232]

(For partial fulfillment of Year II / Semester II in Computer Engineering)

Submitted by

Rajiv Khanal (036129-24)

Alok Dhakal (036115-24)

Navraj Pathak (036137-24)

Sudip Bayalkoti (036110-24)

Submitted to

Mr. Bipesh Subedi

Department of Computer Science and Engineering

Abstract

The BusLedger is a web-based application developed to streamline the management of student transportation records in academic institutions. Traditional methods of maintaining bus route assignments and transport-fee records using paper registers or spreadsheets are often inefficient, error-prone, and difficult to maintain as the number of students increases. This project addresses these challenges by providing a centralized, database-driven solution that enables administrators to efficiently manage student transportation information through a user-friendly web interface.

The system follows a three-tier architecture consisting of a presentation layer built with HTML5, CSS3, and JavaScript, an application layer developed using Node.js and Express.js, and a PostgreSQL relational database for persistent data storage. Secure authentication is implemented using JSON Web Tokens (JWT) and bcrypt for password hashing, while parameterized SQL queries ensure protection against SQL injection attacks. The application supports complete CRUD (Create, Read, Update, Delete) operations, along with search and filtering features based on student name, roll number, department, bus route, and transport-fee status.

Developed as part of the Database Management Systems (DBMS) course, the project demonstrates the practical implementation of core database concepts, including relational schema design, normalization, primary and unique keys, indexing, ENUM data types, triggers, and RESTful API development. The resulting system provides a reliable, secure, and scalable platform for managing student transportation records while reducing manual effort, minimizing data redundancy, and improving the accuracy and accessibility of information.

Table of Contents

1. Introduction

BusLedger is a web-based database application built to digitize and centralize the management of student and faculty transportation records for an academic institution. It replaces manual registers and disconnected spreadsheets with a single relational database accessed through a secure web interface, allowing an administrator to manage departments, bus routes, students, faculty, and transport staff from one place.

The system is built on a three-tier architecture: an HTML/CSS/JavaScript front end, a Node.js/Express.js application layer, and a PostgreSQL relational database. It is developed as a Database Management Systems (DBMS) mini project and is intended to demonstrate practical use of relational schema design, keys, constraints, and normalization rather than to be a full production system.

2. Problem Statement

Institutions that run bus services for students and staff typically track route allocation, departmental affiliation, and transport-fee status using paper registers or independently maintained spreadsheets. This approach has several recurring problems:

- No single source of truth — the same student or route can be entered inconsistently by different staff members.
- No enforced structure — nothing prevents duplicate records, invalid department names, or a bus route being over-assigned beyond its seating capacity.
- Difficult to query — finding all unpaid students on a given route, or all faculty in a department, requires manual searching.
- Poor scalability — as the number of students, faculty, and routes grows, spreadsheets become slow and error-prone to maintain.

BusLedger addresses these problems by moving the data into a normalized relational database where departments, routes, students, faculty, and transport staff are modeled as related tables with enforced constraints.

3. Objectives

- To design a normalized relational database schema for departments, bus routes, students, faculty, and transport staff.
- To enforce data integrity using primary keys, foreign keys, and unique constraints rather than relying on application-level checks alone.
- To model the relationships between students/faculty and their department and bus route explicitly, instead of storing them as free text.
- To provide secure, authenticated administrative access to the system.
- To support CRUD operations and filtered queries (by department, route, or fee status) through the application.

4. How DBMS Solves the Problem

A relational DBMS solves the problems identified above in the following ways:

- Centralized storage: all department, route, student, faculty, and staff data lives in one PostgreSQL database, removing the duplication caused by separate spreadsheets.

- Data integrity through constraints: PRIMARY KEY and UNIQUE constraints (e.g., on roll_no and employee_id) stop duplicate records from being created at the database level.
- Referential integrity through foreign keys: department_id and route_id in the students and faculty tables reference departments and bus_routes, so a student can never be linked to a department or route that does not exist.
- Efficient querying: SQL joins allow the application to answer questions like "list all unpaid students on Route A" directly, instead of scanning spreadsheets manually.
- Concurrent, controlled access: the DBMS manages simultaneous read/write access from multiple administrative sessions safely, which a shared spreadsheet cannot guarantee.

5. DBMS Concepts Used

- Primary keys — every table uses a SERIAL primary key (id) as a unique row identifier.
- Foreign keys — department_id and route_id link students and faculty to departments and bus_routes; route_id links transport_staff to bus_routes.
- Unique constraints — username, dept_name, roll_no, and employee_id are all UNIQUE to prevent duplicate entries.
- NOT NULL constraints — required fields such as name, roll_no, and dept_name cannot be left empty.
- Default values — fee_status defaults to 'unpaid' and created_at defaults to CURRENT_TIMESTAMP.
- Referential integrity — foreign key constraints ensure a student, faculty member, or staff record can never point to a non-existent department or route.
- Joins — INNER/LEFT JOINs are used to combine data across departments, bus_routes, students, faculty, and transport_staff for reporting.
- Normalization — the schema is structured to avoid redundant storage of department and route information (see Section 11).

6. Data Model

The relational (table-based) data model was chosen for this system over alternatives such as a document-oriented (NoSQL) model.

Reasons for choosing the relational model:

- The data is naturally tabular and highly structured — departments, routes, students, faculty, and staff all have a fixed, well-defined set of attributes.
- The relationships between entities (a student belongs to one department and one route; a route can have many students, faculty, and staff) are exactly what foreign keys and joins are designed to express.
- Strong consistency is required — a transport-fee record must always point to a real student, and a student must always point to a real department and route. A relational DBMS enforces this automatically through foreign key constraints, whereas a document model would require the application to enforce it manually.
- The system needs ad-hoc filtering and aggregation (e.g., students per route, unpaid students per department), which SQL supports directly through joins, WHERE clauses, and GROUP BY.
- PostgreSQL provides ACID transactions, which matter for an administrative system where partial or inconsistent updates (e.g., a student left pointing to a deleted route) are unacceptable.

7. Database Schema

The database consists of six tables: users, departments, bus_routes, students, faculty, and transport_staff.

7.1 users

Column	Type	Constraint / Notes
id	SERIAL	PRIMARY KEY
username	VARCHAR(100)	UNIQUE, NOT NULL
password_hash	TEXT	NOT NULL

7.2 departments

Column	Type	Constraint / Notes
id	SERIAL	PRIMARY KEY
dept_name	VARCHAR(100)	UNIQUE, NOT NULL

7.3 bus_routes

Column	Type	Constraint / Notes
id	SERIAL	PRIMARY KEY
route_name	VARCHAR(100)	NOT NULL
bus_number	VARCHAR(20)	—
capacity	INTEGER	—

7.4 students

Column	Type	Constraint / Notes
id	SERIAL	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
roll_no	VARCHAR(50)	UNIQUE, NOT NULL
department_id	INTEGER	FOREIGN KEY → departments(id)
route_id	INTEGER	FOREIGN KEY → bus_routes(id)
fee_status	VARCHAR(20)	DEFAULT 'unpaid'
phone	VARCHAR(20)	—
address	TEXT	—
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

7.5 faculty

Column	Type	Constraint / Notes
id	SERIAL	PRIMARY KEY
faculty_name	VARCHAR(100)	NOT NULL
employee_id	VARCHAR(50)	UNIQUE, NOT NULL

Column	Type	Constraint / Notes
department_id	INTEGER	FOREIGN KEY → departments(id)
route_id	INTEGER	FOREIGN KEY → bus_routes(id)
designation	VARCHAR(50)	—

7.6 transport_staff

Column	Type	Constraint / Notes
id	SERIAL	PRIMARY KEY
staff_name	VARCHAR(100)	NOT NULL
role	VARCHAR(50)	e.g., Driver, Conductor
route_id	INTEGER	FOREIGN KEY → bus_routes(id)
phone	VARCHAR(20)	—

8. Keys and Relationships (ER Diagram)

Entity relationships in the schema:

- departments (1) — (M) students : one department has many students, via students.department_id.
- departments (1) — (M) faculty : one department has many faculty members, via faculty.department_id.
- bus_routes (1) — (M) students : one route carries many students, via students.route_id.
- bus_routes (1) — (M) faculty : one route carries many faculty, via faculty.route_id.
- bus_routes (1) — (M) transport_staff : one route has many assigned staff (driver/conductor), via transport_staff.route_id.
- users is a standalone table for administrator authentication and has no foreign-key relationship to the other tables.

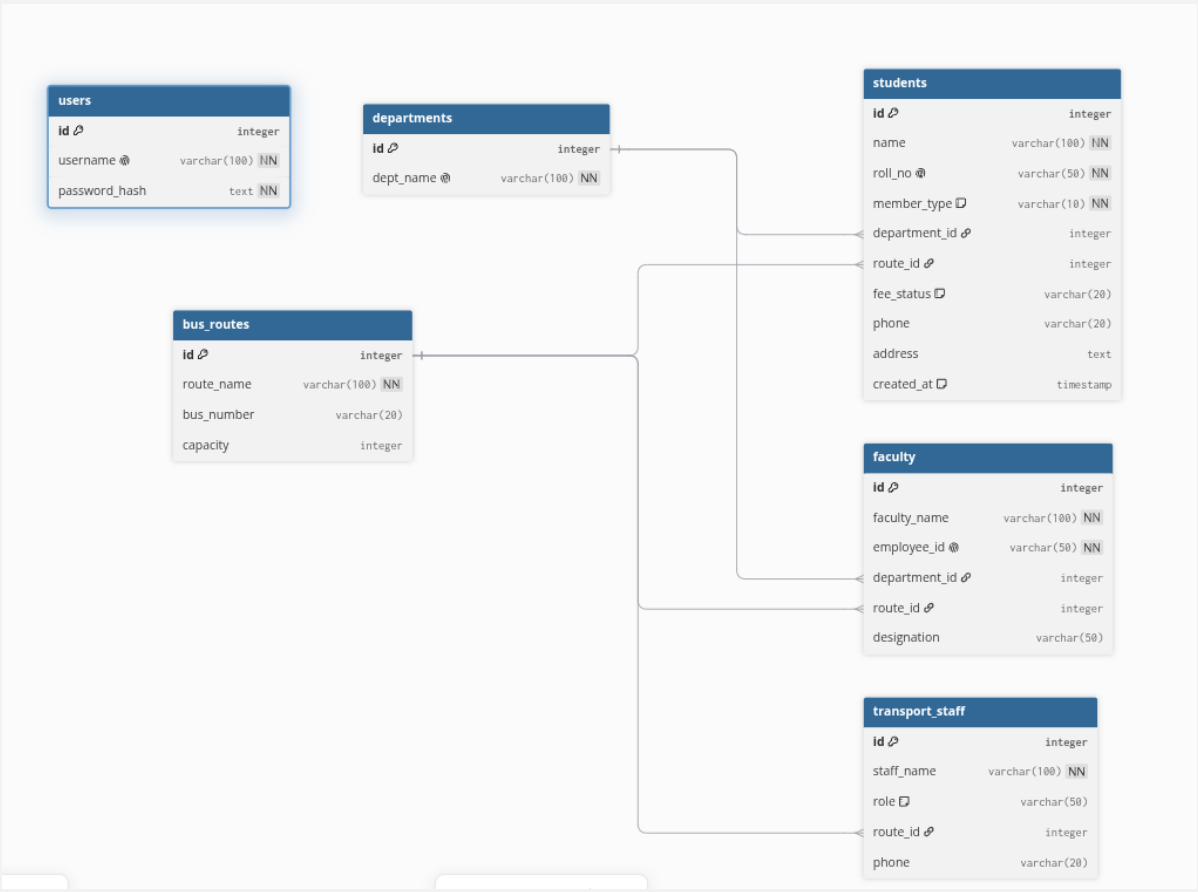

Figure 8.1 — Entity Relationship (ER) Diagram

NEEDED: Insert the actual ER diagram (e.g., drawn in draw.io / dbdiagram.io) showing all six tables, their primary/foreign keys, and cardinalities described above.

9. Relational Schema

The relational schema, showing each table's attributes with primary keys (PK) and foreign keys (FK):

```
users(id PK, username, password_hash)
```

```
departments(id PK, dept_name)
bus_routes(id PK, route_name, bus_number, capacity)
students(id PK, name, roll_no, department_id FK→departments.id,
          route_id FK→bus_routes.id, fee_status, phone,
          address, created_at)
faculty(id PK, faculty_name, employee_id,
          department_id FK→departments.id,
          route_id FK→bus_routes.id, designation)
transport_staff(id PK, staff_name, role,
          route_id FK→bus_routes.id, phone)
```

10. Normalisation

The schema was designed directly in normalized form rather than being normalized after the fact:

- 1NF: every column holds a single atomic value (e.g., no comma-separated lists of departments or routes in one field), and every table has a primary key.
- 2NF: all tables have a single-column primary key (id), so there are no partial dependencies to remove.
- 3NF: non-key attributes depend only on the primary key. Department and route details (dept_name, route_name, bus_number, capacity) are stored once in departments and bus_routes and referenced by department_id / route_id elsewhere, instead of being repeated on every student, faculty, or staff row. This removes the transitive dependency and redundancy that would exist if, for example, dept_name were stored directly on the students table.

Because department and route information is stored exactly once and referenced by foreign key, updating a department name or a bus route's capacity requires changing a single row rather than every student/faculty row that references it. No further normalization is required for this schema.

11. System Overview

At a high level, the system follows this flow: an administrator logs in, is issued a session/token, and then performs CRUD operations on departments, bus routes, students, faculty, and transport staff through the dashboard. Each operation is translated by the application layer into a parameterized SQL statement executed against PostgreSQL.

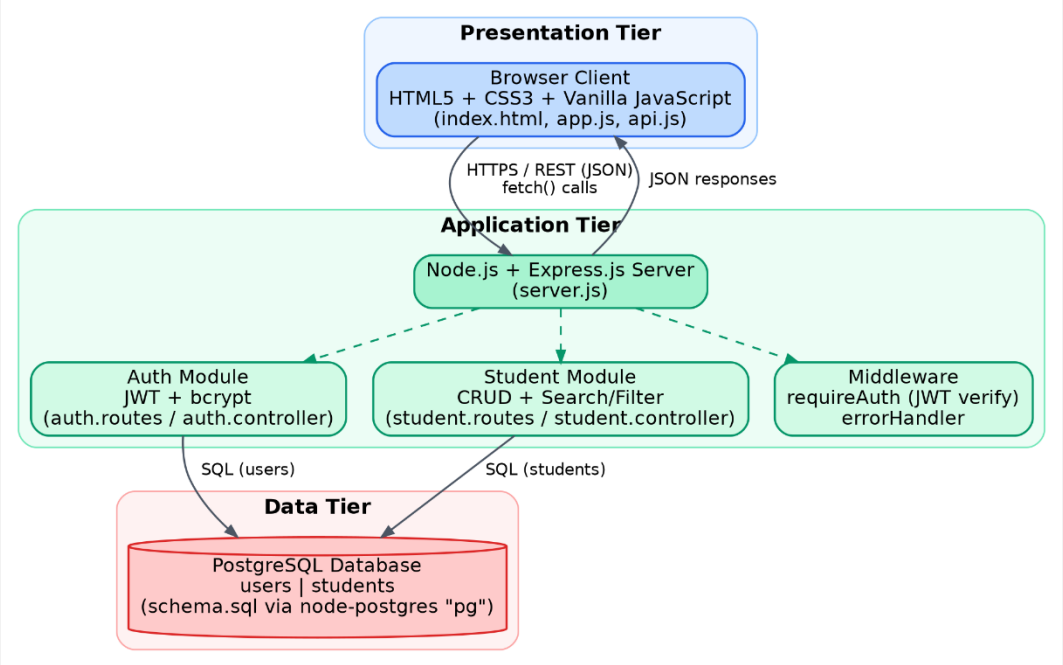

Figure 11.1 — System Architecture / Flowchart

NEEDED: Insert a diagram showing the three-tier flow: Front end → Express.js API (with auth middleware) → PostgreSQL database, and/or a flowchart of the login → dashboard → CRUD process.

12. Outcomes

At the completion of this project, the following outcomes were achieved:

- A fully normalized six-table relational schema modeling departments, bus routes, students, faculty, and transport staff, with enforced primary/foreign key and uniqueness constraints.
- A working web application supporting authenticated CRUD operations on all entities in the schema.
- The ability to query and filter records across related tables (e.g., students by department or route, unpaid fees by route) using SQL joins.
- Practical, hands-on experience applying primary keys, foreign keys, constraints, and normalization principles to a real schema design problem.

13. Sample Queries

The following are representative SQL queries supported by the schema (maximum 10):

```
1. List all students with their department and route names:

SELECT s.name, s.roll_no, d.dept_name, r.route_name
FROM students s
JOIN departments d ON s.department_id = d.id
JOIN bus_routes r ON s.route_id = r.id;
```

```
2. Find all students with unpaid fees:

SELECT name, roll_no, fee_status FROM students
WHERE fee_status = 'unpaid';
```

3. Count students assigned to each bus route:

```
SELECT r.route_name, COUNT(s.id) AS total_students
FROM bus_routes r
LEFT JOIN students s ON s.route_id = r.id
GROUP BY r.route_name;
```

4. List all faculty in the Computer Science department:

```
SELECT f.faculty_name, f.designation
FROM faculty f
JOIN departments d ON f.department_id = d.id
WHERE d.dept_name = 'Computer Science';
```

5. Find drivers/conductors assigned to a given route:

```
SELECT staff_name, role, phone FROM transport_staff
WHERE route_id = (SELECT id FROM bus_routes
                  WHERE route_name = 'Route A - North');
```

6. Check remaining seat capacity on each route:

```
SELECT r.route_name, r.capacity, COUNT(s.id) AS occupied,
       r.capacity - COUNT(s.id) AS seats_left
FROM bus_routes r
LEFT JOIN students s ON s.route_id = r.id
GROUP BY r.route_name, r.capacity;
```

7. Add a new student record:

```
INSERT INTO students (name, roll_no, department_id,
  route_id, phone, address)
VALUES ('Aayush Rai', 'CS-2024-041', 1, 2,
        '98XXXXXXXX', 'Dhulikhel, Kavre');
```

8. Update a student's fee status to paid:

```
UPDATE students SET fee_status = 'paid'
WHERE roll_no = 'CS-2024-041';
```

9. Remove a student record:

```
DELETE FROM students WHERE roll_no = 'CS-2024-041';
```

10. List every department with its student and faculty counts:

```
SELECT d.dept_name,
  COUNT(DISTINCT s.id) AS student_count,
  COUNT(DISTINCT f.id) AS faculty_count
FROM departments d
LEFT JOIN students s ON s.department_id = d.id
LEFT JOIN faculty f ON f.department_id = d.id
GROUP BY d.dept_name;
```

13.1 Query Used ScreenShots

```
admin=# CREATE USER busstudentdbuser WITH PASSWORD 'passwordforbusdb';
CREATE ROLE
admin=# create database busstudentdb;
CREATE DATABASE
admin=# alter database busstudentdb owner to busstudentdbuser;
ALTER DATABASE
admin=#
```

```
admin=# \c busstudentdb busstudentdbuser
You are now connected to database "busstudentdb" as user "busstudentdbuser".
busstudentdb=> SELECT current_database(), current_user;
 current_database |   current_user
------------------+------------------
 busstudentdb     | busstudentdbuser
(1 row)
```

```
admin=# \c busstudentdb busstudentdbuser
You are now connected to database "busstudentdb" as user "busstudentdbuser".
busstudentdb=> CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(100) UNIQUE NOT NULL,
    password_hash TEXT NOT NULL
);

CREATE TABLE departments (
    id SERIAL PRIMARY KEY,
    dept_name VARCHAR(100) UNIQUE NOT NULL
);
CREATE TABLE
CREATE TABLE
busstudentdb=> CREATE TABLE bus_routes (
    id SERIAL PRIMARY KEY,
    route_name VARCHAR(100) NOT NULL,
    bus_number VARCHAR(20),
    capacity INTEGER
);
CREATE TABLE
busstudentdb=>
```

```
busstudentdb=> CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    roll_no VARCHAR(50) UNIQUE NOT NULL,
    member_type VARCHAR(10) NOT NULL DEFAULT 'student' CHECK (member_type IN ('student', 'staff')),
    department_id INTEGER REFERENCES departments(id),
    route_id INTEGER REFERENCES bus_routes(id),
    fee_status VARCHAR(20) DEFAULT 'unpaid',
    phone VARCHAR(20),
    address TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE
busstudentdb=>
```

```
busstudentdb=> CREATE TABLE faculty (
    id SERIAL PRIMARY KEY,
    faculty_name VARCHAR(100) NOT NULL,
    employee_id VARCHAR(50) UNIQUE NOT NULL,
    department_id INTEGER REFERENCES departments(id),
    route_id INTEGER REFERENCES bus_routes(id),
    designation VARCHAR(50)
);
CREATE TABLE
busstudentdb=>
```

```
busstudentdb=> CREATE TABLE transport_staff (
    id SERIAL PRIMARY KEY,
    staff_name VARCHAR(100) NOT NULL,
    role VARCHAR(50), -- e.g., Driver, Conductor
    route_id INTEGER REFERENCES bus_routes(id),
    phone VARCHAR(20)
);
CREATE TABLE
busstudentdb=>
```

```
busstudentdb=> INSERT INTO departments (dept_name) VALUES
    ('Computer Science'),
    ('Mechanical'),
    ('Civil'),
    ('Electrical'),
    ('Electronics & Communication'),
    ('Architecture'),
busstudentdb=> INSERT INTO bus_routes (route_name, bus_number, capacity) VALUES
    ('Route A - Dhulikhel', 'B-01', 40),
    ('Route B - Banepa', 'B-02', 40),
    ('Route C - Panauti', 'B-03', 32),
    ('Route D - Panchkhal', 'B-04', 32),
    ('Route E - Nala', 'B-05', 28),
    ('Route F - Dhungkharka', 'B-06', 28),
    ('Route G - Namobuddha', 'B-07', 24),
    ('Route H - Khopasi', 'B-08', 24),
    ('Route I - Mahadevsthan', 'B-09', 24),
    ('Route J - Sathighar Bhagawati', 'B-10', 20);
INSERT 0 10
busstudentdb=>
```

14. Screenshots of the System

The following pages present screenshots of the working system.

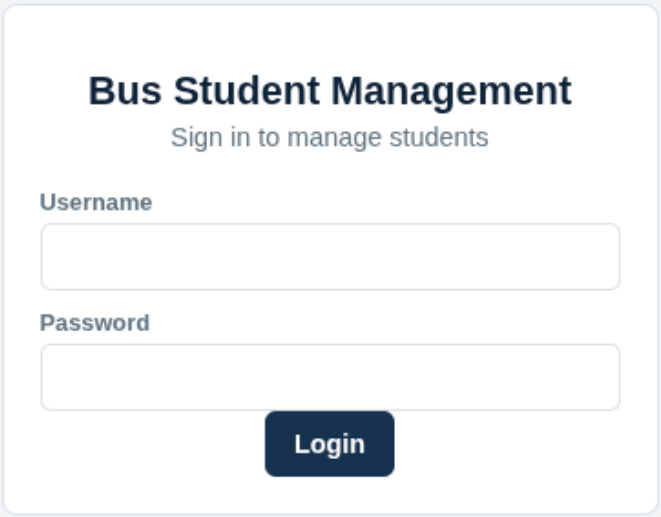

Figure 14.1 — Login Page

NEEDED: Screenshot of the administrator login screen.

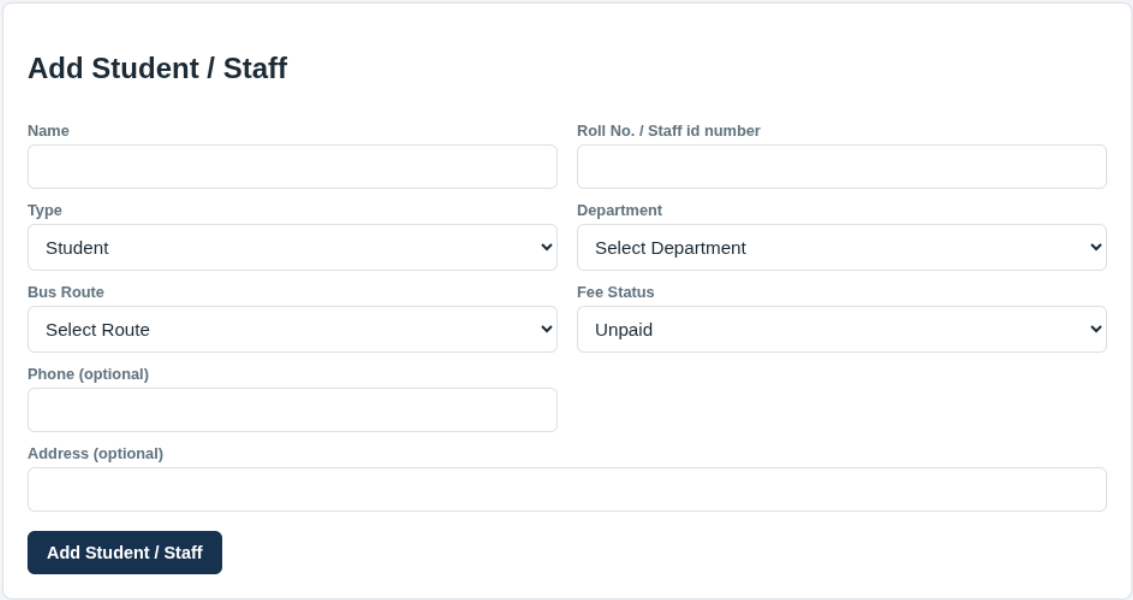

Figure 14.2 — Dashboard / Student List View

NEEDED: Screenshot of the main dashboard showing the list of students with filters.

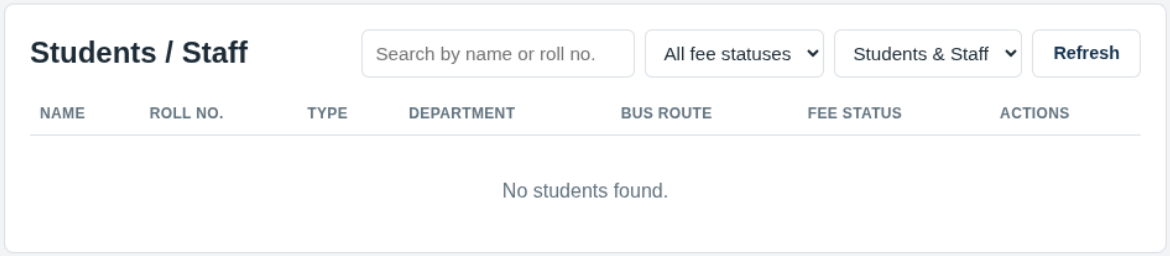

Figure 14.3 — Add / Edit Record Form

NEEDED: Screenshot of the form used to add or edit a student, faculty, or route record.

15. Conclusion

BusLedger demonstrates the practical application of core DBMS concepts — primary and foreign keys, unique and not-null constraints, referential integrity, joins, and normalization — to a real institutional problem. By modeling departments, bus routes, students, faculty, and transport staff as related tables instead of flat, free-text records, the system removes the redundancy and inconsistency inherent in spreadsheet-based tracking, and enforces data integrity directly at the database level. The project met its stated objectives of designing a normalized schema, enforcing data integrity through constraints, and supporting authenticated, queryable access to institutional transport data.